

Lab: Event-Driven Nginx Log Monitoring with EDA

About this Lab

This lab demonstrates how to build a self-healing automation pipeline that monitors Nginx logs for specific error patterns and automatically triggers remediation. Using rsyslog (native to RHEL) to forward log entries via HTTP to an EDA Event Stream, the EDA Controller evaluates incoming events against a rulebook and triggers an Automation Controller job template to restart the Nginx service when a critical error is detected. No additional agents or message brokers are required.

Learning Objectives

After completing this module, you will be able to:

- Configure an EDA Rulebook that consumes events from an Event Stream
- Create an EDA Event Stream to provide an endpoint for external log sources
- Create a remediation playbook and Job Template to restart the Nginx service
- Test the full event-driven automation pipeline end-to-end
- Implement the throttling feature on rulebooks
- Optional: Enhance the solution so that EDA determines the target host on which to restart the Nginx service directly from the event payload

Architecture Overview

The following diagram illustrates the flow of events through the system:

```
Nginx --> /var/log/nginx/ --> rsyslog --> Event Stream (receives the event)
--> (event passed to) rulebook activation -> run_job_template --> Restart Nginx
```

1. **Nginx** writes errors to its log file
2. **rsyslog** watches the log file (imfile) and forwards matching entries as JSON via HTTP (omhttp) to the EDA Event Stream
3. **EDA Controller** receives the event via the Event Stream, routes it to the Rulebook Activation, and evaluates it against the rulebook conditions
4. **Job Template:** when the condition matches, EDA triggers a Job Template in Automation Controller to restart Nginx

Prerequisites

- Access to the shared **Ansible Automation Platform (AAP)** environment (Controller, EDA, Hub)
- A RHEL 9.4+ target host registered to a Satellite, the Red Hat CDN or equivalent.
- A Git repository for your playbooks and rulebooks
- Network connectivity from the Linux target host to the AAP gateway
- AAP credential configured

 **Admin Prerequisite:** Before proceeding with this lab, a user with **System Administrator** privileges must enable the following setting: navigate to **UI → Settings → Platform Gateway settings → Allow external users to create OAuth2 tokens**, set it to **Enabled**, and click **Save**. This is required for lab participants to create OAuth2 tokens during the lab.

Step 0: Create the Basic Event Stream Credential

In the GUI, navigate to

In the GUI, navigate to **Automation Decisions → Infrastructure → Credentials** and click **Create credential**. Fill in the following fields:

- **Name:** <username>_basic_demo_cred (replace <username> with your actual username)
- **Organization:** select your Organization
- **Credential type:** Basic Event Stream

Under Type Details, specify a username and password:

- **Username:** <username>
- **Password:** <password>

Important

These values must match the `eda_basic_user` and `eda_basic_pass` variables that will be passed to the Job Template in Step 4.

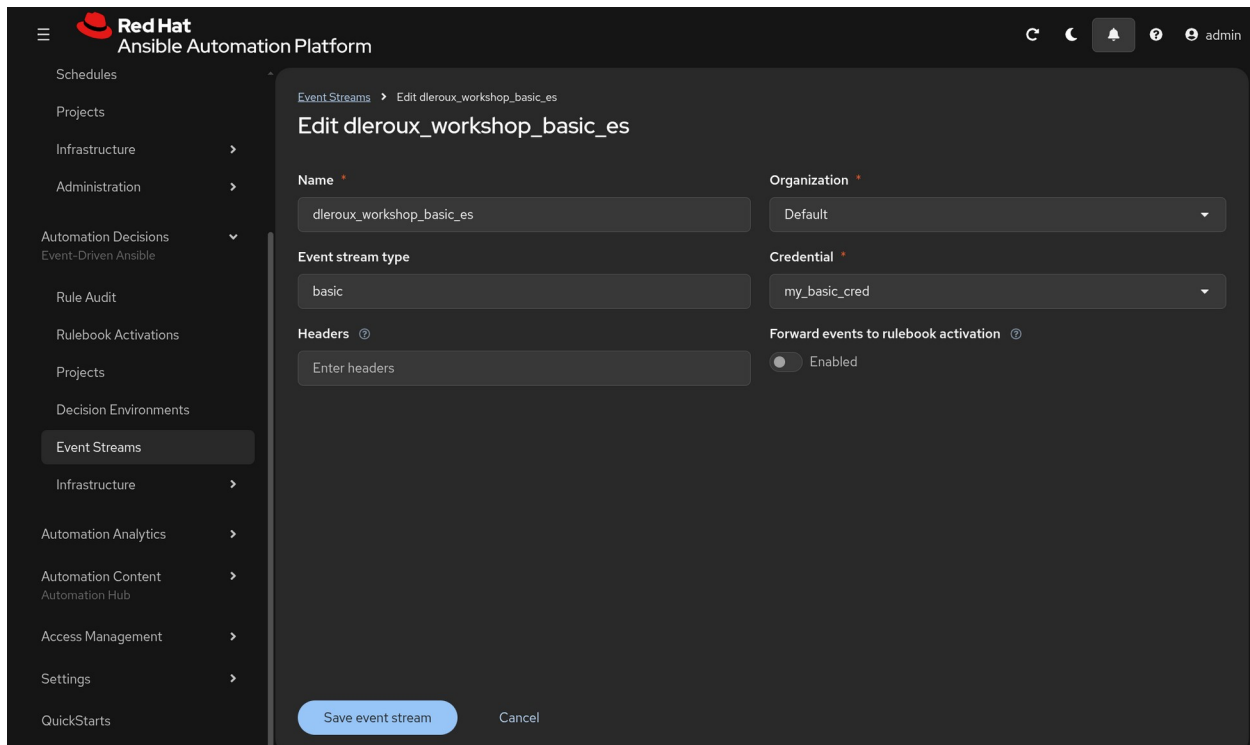
The screenshot shows the Red Hat Ansible Automation Platform interface. On the left is a navigation sidebar with categories like Schedules, Projects, Infrastructure, Administration, Automation Decisions, Rule Audit, Rulebook Activations, Projects, Decision Environments, Event Streams, Infrastructure, Credentials (highlighted), Credential Types, Automation Analytics, Automation Content, and Access Management. The main panel displays the 'Edit dleroux_my_basic_cred' form. The form has the following fields: 'Name' (dleroux_my_basic_cred), 'Description' (placeholder: Enter description), 'Organization' (Default), 'Credential type' (Basic Event Stream), 'Type Details' section containing 'Username' (username) and 'Password' (password). At the bottom are 'Save credential' and 'Cancel' buttons.

Example: Basic Event Stream credential creation

Step 1: Create the Event Stream

In the GUI, navigate to **Automation Decisions** → **Event Streams** and click **Create event stream**. Fill in the following fields:

- **Name:** <username>_workshop_basic_es
- **Organization:** select your Organization
- **Event stream type:** Basic
- **Credential:** select the credential created in Step 0



Example: Event Stream creation

Note

Leave **Forward events to rulebook activation** disabled.

Step 2: Create an Inventory and Credential to Connect to your RHEL 9 VM

2.1 Create the Inventory

In the GUI, navigate to **Automation Execution** → **Infrastructure** → **Inventories** and click **Create inventory**. Fill in the following fields:

- **Name:** <username>_my_rhel9vm
- **Organization:** select your Organization

Once created, navigate to the **Hosts** tab and click **Create host**. Enter either the FQDN or the IP address of the RHEL 9 VM you will be working with.

2.2 Create the Machine Credential

In the GUI, navigate to **Automation Execution** → **Infrastructure** → **Credentials** and click **Create credential**. Fill in the following fields:

- **Name:** <username>_rhel9vm_cred
- **Organization:** select your Organization

- **Credential type:** Machine

Note

Make sure **Privilege Escalation** is properly configured for the username connecting to your RHEL 9 VM (e.g. escalation method: `sudo`, escalation username: `root`).

Here is an example of what the credential and inventory should look like:

The screenshot shows the 'Edit dleroux_rhel9vm_cred' page in the Red Hat Ansible Automation Platform. The 'Credential type' is set to 'Machine'. The 'Username' is 'ansible' and the 'Password' is 'ENCRYPTED'. The 'Privilege Escalation Method' is 'sudo' and the 'Privilege Escalation Username' is 'root'. The 'Privilege Escalation Password' is 'ENCRYPTED'. The 'Save credential' button is highlighted in blue.

Example: Machine credential with Privilege Escalation configured

The screenshot shows the 'dleroux_my_rhel9vm' inventory page. The 'Hosts' tab is selected. A table lists the hosts with columns for Name, Description, and Related groups. The host 'dleroux-r2vmware.tamlab.rdu2.redhat.com' is listed. The 'Create host' button is highlighted in blue.

Example: Inventory with RHEL 9 host

Step 3: Create a Project for the Playbooks

In the GUI, navigate to **Automation Execution** → **Projects** and click **+ Create project**. Fill in the following fields:

- **Name:** <username>_basic_eda_playbooks
- **Organization:** select your Organization
- **Source control type:** Git
- **Source control URL:** https://github.com/daleroux/basic_eda

Once complete, click **Save project** and wait for the project to finish synchronizing before proceeding to the next step.

Step 4: Create the Job Template to Prepare the RHEL 9 VM

This job template will deploy nginx and configure the RHEL 9 VM so it forwards nginx error logs to EDA.

In the GUI, navigate to **Automation Execution → Templates** and click **+ Create template**. Fill in the following fields:

- **Name:** <username>_deploy_nginx_eda_logging_pipeline
- **Job type:** Run
- **Inventory:** <username>_my_rhel9vm
- **Project:** <username>_basic_eda_playbooks
- **Playbook:** playbooks/deploy_nginx_eda_logging_pipeline.yml
- **Execution environment:** Default execution environment
- **Credentials:** <username>_rhel9vm_cred
- **Options:** check **Privilege escalation**

Under Extra Variables, set the following:

```
aap_fqdn: your_gateway_fqdn.com
eda_UUID: 92ac95b7-a078-4114-aa9b-378184d18b80
eda_basic_user: <your_username>
eda_basic_pass: <your_password>
```

Important

Before saving, update the Extra Variables to match your environment:

- Replace `aap_fqdn` with your AAP Gateway FQDN
- Replace `eda_UUID` with your Event Stream UUID (found on the Event Stream details page created in Step 1)
- Replace `eda_basic_user` and `eda_basic_pass` with the values you set in Step 0

Once saved, **run the job template** and verify it completes successfully before proceeding to the next step.

Step 5: Simulate an nginx Error on your RHEL 9 System

Log into your RHEL 9 system using the service account configured with passwordless sudo privilege escalation and run the following command to inject a simulated nginx error into the log file:

```
sudo tee -a /var/log/nginx/error.log <<< '[error] upstream timed out (110: Connection timed out) while reading response header from upstream'
```

In the GUI, navigate to **Automation Decisions → Event Streams** and select `<username>_workshop_basic_es`. Verify that the payload was received by checking the **Events received** counter.

Important

Only proceed to the next step once you have confirmed the payload is showing up at the Event Stream level. This confirms that the nginx log forwarding deployed in Step 4 is working correctly and that the Event Stream is ready to be associated with a rulebook.

If the previous `sudo tee` command did not create an event in the Event Stream, use the following commands to help debug the issue.

First, generate the Base64-encoded value for your Event Stream username and password:

```
printf '%s' 'your_eda_basic_user:your_eda_basic_pass' | base64
```

Compare the output with the `AUTH_B64` value in the forwarding script. You can view the configured value with:

```
sudo grep '^AUTH_B64=' /usr/local/bin/eda_event_forwarder.sh
```

If the Base64 value does not match, the script was generated with different credentials. In that case, rerun the Step 4 job template with the correct `eda_basic_user` and `eda_basic_pass` values.

If the Base64 value matches, proceed with the following manual test:

```
cat <<'EOF' | sudo /usr/local/bin/eda_event_forwarder.sh
{"message":"[error] upstream timed out (110: Connection timed out) while
reading response header from upstream","hostname":"manual-
test","source":"manual-test"}
EOF
```

After running the manual test, check the Event Stream in AAP and confirm that the Events received counter increased.

Step 6: Create a Project for the Rulebooks

In the GUI, navigate to **Automation Decisions → Projects** and click **+ Create project**. Fill in the following fields:

- **Name:** `<username>_basic_eda_project`
- **Organization:** select your Organization
- **Source control type:** Git

- **Source control URL:** https://github.com/daleroux/basic_eda

Once complete, click **Create project** and wait for the project to finish synchronizing before proceeding to the next step.

Note

This project points to the same GitHub repository as the one created in Step 3, but is created under **Automation Decisions** rather than **Automation Execution** — rulebooks and playbooks are managed by separate project resources in AAP.

Step 7: Create a Rulebook Activation that Prints its Payload

In the GUI, navigate to **Automation Decisions → Rulebook Activations** and click **+ Create rulebook activation**. Fill in the following fields:

- **Name:** <username>_basic_eda_rulebook
- **Organization:** select your Organization
- **Project:** <username>_basic_eda_project
- **Rulebook:** print_payload.yml
- **Event streams:** associate <username>_workshop_basic_es to the rulebook
- **Decision environment:** Default Decision Environment
- **Rulebook activation enabled?:** on

Keep the default options for all other fields, then click **Create rulebook activation**.

7.1 Trigger the Rulebook

Log into your RHEL 9 system using the service account configured with passwordless sudo privilege escalation and run the following command:

```
sudo tee -a /var/log/nginx/error.log <<< '[error] upstream timed out (110: Connection timed out) while reading response header from upstream'
```

7.2 Verify the Results

In the GUI, navigate to the rulebook activation you just created and confirm that the **Firecount** has increased from 0 to 1.

Note

If the Firecount did not increase, double-check the following before proceeding:

- The Event Stream <username>_workshop_basic_es is correctly associated with this rulebook activation
- The payload was confirmed as received at the Event Stream level in Step 5

- The rulebook activation status is **Running**

Important

Printing and verifying the payload at both the Event Stream level (Step 5) and the Rulebook level (this step) requires a few additional steps, but ensures the solution is built on a verified and solid foundation before introducing automation actions in the steps ahead.

Step 8: Create an OAuth Token for your User

In the GUI, navigate to **Access Management → Users** and select your username. Click the **Tokens** tab, then click **+ Create token**. Fill in the following fields:

- **OAuth Application:** leave empty
- **Scope:** `write`

Click **Create token**.

Important

Your token will only be displayed **once**. Make sure to copy and save it somewhere secure before closing this dialog — it cannot be retrieved afterward.

Step 9: Create a Red Hat Ansible Automation Platform Credential

This credential allows rulebooks running in EDA to authenticate with the AAP Controller and launch Job Templates for remediation actions.

In the GUI, navigate to **Automation Decisions → Infrastructure → Credentials** and click **+ Create credential**. Fill in the following fields:

- **Name:** `<username>_aap_credential`
- **Organization:** select your Organization
- **Credential type:** `Red Hat Ansible Automation Platform`
- **Red Hat Ansible Automation Platform:**
`https://your_gateway_fqdn.com/api/controller/`
- **OAuth Token:** paste the token copied in Step 8
- **Verify SSL:** uncheck

Important

Make sure to use the OAuth token generated in Step 8. If you did not save it at the time, you

will need to return to **Access Management → Users** and generate a new one.

Once complete, click **Save credential**.

Step 10: Create the Remediation Job Template

This Job Template will be triggered by EDA rulebooks to restart nginx on your RHEL 9 VM whenever a matching error condition is detected in the logs.

In the GUI, navigate to **Automation Execution → Templates** and click **+ Create template**. Fill in the following fields:

- **Name:** nginx_restart
- **Job type:** Run
- **Inventory:** <username>_my_rhel9vm
- **Project:** <username>_basic_eda_playbooks
- **Playbook:** playbooks/nginx_restart.yml
- **Execution environment:** Default execution environment
- **Credentials:** <username>_rhel9vm_cred
- **Options:** check **Privilege escalation**

Once complete, click **Create job template**.

Note

Unlike the Job Template created in Step 4, this one requires no Extra Variables — it simply restarts nginx on the target host. It will be referenced by name (nginx_restart) in the rulebooks used in the steps ahead.

Note this job template name is not prefixed by your username

Step 11: Create a Rulebook Activation that Restarts nginx on Error

In the GUI, navigate to **Automation Decisions → Rulebook Activations** and click **+ Create rulebook activation**. Fill in the following fields:

- **Name:** <username>_check_nginx_logs
- **Organization:** select your Organization
- **Project:** <username>_basic_eda_project
- **Rulebook:** check_nginx_logs.yml
- **Event streams:** associate <username>_workshop_basic_es to the rulebook

- **Credential:** <username>_aap_credential
- **Decision environment:** Default Decision Environment
- **Rulebook activation enabled?:** on

Under Variables, set the following:

```
target_organization: "Your_Selected_Organization"
```

Keep the default options for all other fields, then click **Create rulebook activation**.

On your RHEL 9 system using the service account configured with passwordless sudo privilege escalation, run the following command:

```
sudo tee -a /var/log/nginx/error.log <<< '[error] upstream timed out (110: Connection timed out) while reading response header from upstream'
```

Note

This action should have launched the `nginx_restart` Job Template. Verify under **Automation Execution → Jobs** that the job was triggered and completed successfully.

On your RHEL 9 VM, run the following command to confirm nginx was restarted:

```
sudo systemctl status nginx
```

Verify that the Active line shows the service was restarted recently.

 **Note:** Before proceeding to Step 12, make sure to **deactivate the Rulebook Activation created in Step 11**. Both activations listen on the same port (5005), and having two active at the same time will cause a conflict.

Step 12: Create a Rulebook Activation with Throttling (once every 10 minutes)

In this step, we will create a new rulebook activation based on an improved rulebook that fires the restart action only once per 10-minute window, with throttling tracked per hostname (event.payload.hostname).

12.1 Create the Rulebook Activation

In the GUI, navigate to **Automation Decisions → Rulebook Activations** and click **+ Create rulebook activation**. Fill in the following fields:

- **Name:** <username>_throttle_nginx_logs
- **Organization:** select your Organization
- **Project:** <username>_basic_eda_project
- **Rulebook:** throttle_nginx_logs.yml
- **Event streams:** associate <username>_workshop_basic_es to the rulebook
- **Credential:** <username>_aap_credential
- **Decision environment:** Default Decision Environment
- **Rulebook activation enabled?:** on

Under Variables, set the following:

```
target_organization: "Your_Selected_Organization"
```

Keep the default options for all other fields, then click **Create rulebook activation**.

12.2 Trigger the Rulebook (Simulate nginx Errors)

On your RHEL 9 system, using the service account configured with passwordless sudo privilege escalation, run the following command twice in a row:

```
sudo tee -a /var/log/nginx/error.log <<< '[error] upstream timed out (110:
Connection timed out) while reading response header from upstream'
```

Run it a second time:

```
sudo tee -a /var/log/nginx/error.log <<< '[error] upstream timed out (110:
Connection timed out) while reading response header from upstream'
```

12.3 Expected Behavior

Note

Even though the error was injected into the log **twice**, the `nginx_restart` Job Template should have been triggered **only once**. This confirms that the throttling configured in `throttle_nginx_logs.yml` is working as expected.

To verify:

- The **Firecount** for this rulebook activation should show **1** (the rule fired only once)
- The **Events received** counter at the Event Stream level should show **2** (both log entries were received and processed)

Optional Exercises

A: Role-Based Access Control for EDA

Create a Team within your Organization and configure it with the following permissions:

- **Rulebook Activations:** Create
- **Projects:** Create
- **Decision Environments:** View/Use only (cannot toggle forwarding or disable)
- **Event Streams:** View/Use only (cannot toggle forwarding to a rulebook activation or disable it)

Then create a User, add them as a member of this team, and verify the permissions are working as expected by attempting to recreate the EDA project under a different name, along with a new rulebook activation.

B: Pass the Hostname FQDN in the Payload Sent to EDA

Modify the logging sent to EDA so that the hostname FQDN is:

5. Included in the payload received at the Event Stream level
6. Passed through to the rulebook as part of the event data
7. Forwarded to the Job Template so that the `nginx_restart` playbook targets only the specific host that triggered the alert — rather than all hosts in the inventory

C: Apply an Event Filter to Transform the Payload into JSON

Try applying an **Event Filter** directly to the source within your rulebook. The goal is to use the ***ansible.eda.json_filter*** (or another filter of your choice). Verify if your rule conditions still evaluate correctly after the filter is applied, and observe how the structured payload differs from the string received without the filter.